CY-4973/7790

Kernel Security: how2rootkit

AArch64 Virtual Memory: From Page Tables to Syscall Hooks

Agenda

1. **motivation**: Why are page tables the way they are?
    1. why a flat page table is absurd, and how splitting the VPN across levels solves it
2. implementing a `resolve_pte()`:
    1. PTE_RDONLY, and the PMD section cast
3. **TLB**

What Does Virtual Memory Need to Do?

- Isolation: every process gets its own address space with some optionally shared regions. Process A's pointer `0x4000_0000` maps to different physical RAM than Process B's `0x4000_0000`.
- Privilege separation: the kernel's memory is invisible and inaccessible from userspace. A user process cannot read or write kernel data structures.
- Sparse allocation: a process's address space is mostly empty. Code sits near the bottom, stack near the top, everything in between is unmapped. The hardware must handle this efficiently.

Mechanism: a per-process data structure that translates virtual addresses to physical addresses, checked on every memory access by **someone**

MMU

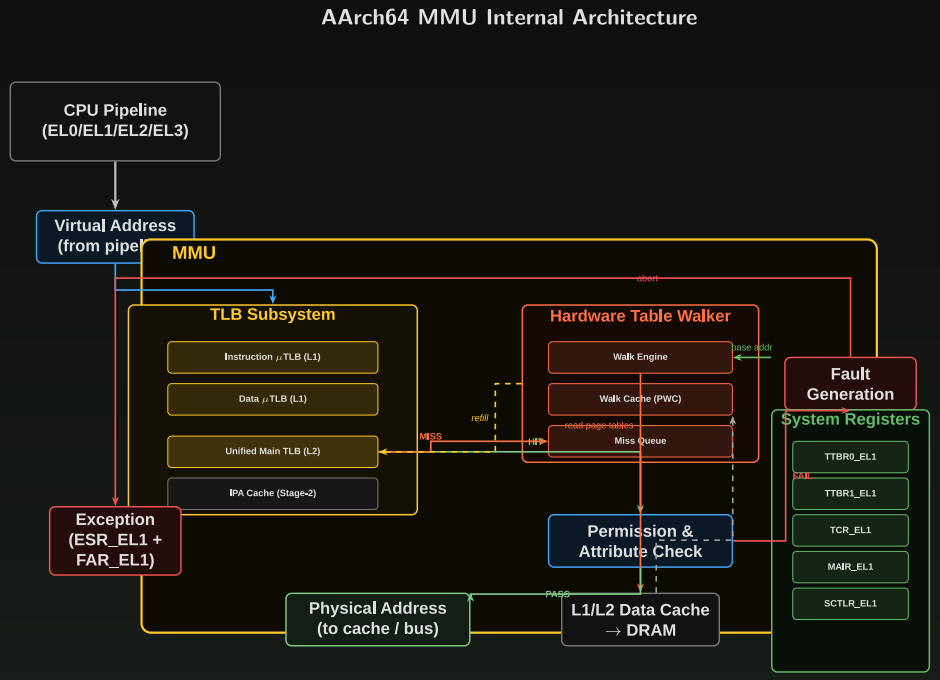
The **Memory Management Unit** is a hardware block inside every AArch64 core. Every memory access passes through it.

The MMU does three things:

1. **Address Translation** --- converts VA to PA (enables per-process isolation)
2. **Permission Checking** --- verifies AP/PXN/UXN against current EL
3. **Memory Attribute Assignment** --- cacheability, shareability, device/normal type

Each core has its own MMU. Controlled via system registers (`MRS`/`MSR`). Disabled at reset (`SCTLR_EL1.M = 0`).

What the MMU Is



The **TLB** caches completed translations. The **Table Walker** performs multi-level page table reads from DRAM on TLB miss. **Permission Check** verifies AP/PXN/UXN bits against the current exception level.

High level

- logically organize physical memory into blocks
- Different architectures refer to these blocks by different names (Physical page, Physical Frame ...etc)
- It is the smallest "granule" for which we can allocate and protect
- On Aarch64, the default size is 4 kilobytes (2^{12} bytes)
- Addresses are Translated by subdividing a Virtual Address into
    - a Physical Frame Number
    - An offset into that Frame

Sanity Check

- Given a page size of 4KB, how many bits are required to address every byte within the page?
-
-

Sanity Check

- Given a page size of 4KB, how many bits are required to address every byte within the page?
- $\log_2(4096) = \log_2(2^{12}) =$
-

Sanity Check

- Given a page size of 4KB, how many bits are required to address every byte within the page?
- $\log_2(4096) = \log_2(2^{12}) =$
- $\log_2(4096) = \log_2(2^{12}) = 12$

Sanity Check

- Given a page size of 16KB, how many bits are required to address every byte within the page?
-
-

Sanity Check

- Given a page size of 16KB, how many bits are required to address every byte within the page?
- $\log_2(2^{14}) =$
-

Sanity Check

- Given a page size of 16KB, how many bits are required to address every byte within the page?
- $\log_2(2^{14}) =$
- $\log_2(2^{14}) = 14$

Sanity Check

- Given a page size of 64KB, how many bits are required to address every byte within the page?
-
-

Sanity Check

- Given a page size of 64KB, how many bits are required to address every byte within the page?
- $\log_2(2^{16}) =$
-

Sanity Check

- Given a page size of 64KB, how many bits are required to address every byte within the page?
- $\log_2(2^{16}) =$
- $\log_2(2^{16}) = 16$

Attempt 1: The Flat Page Table

The simplest design: one array entry per virtual page.

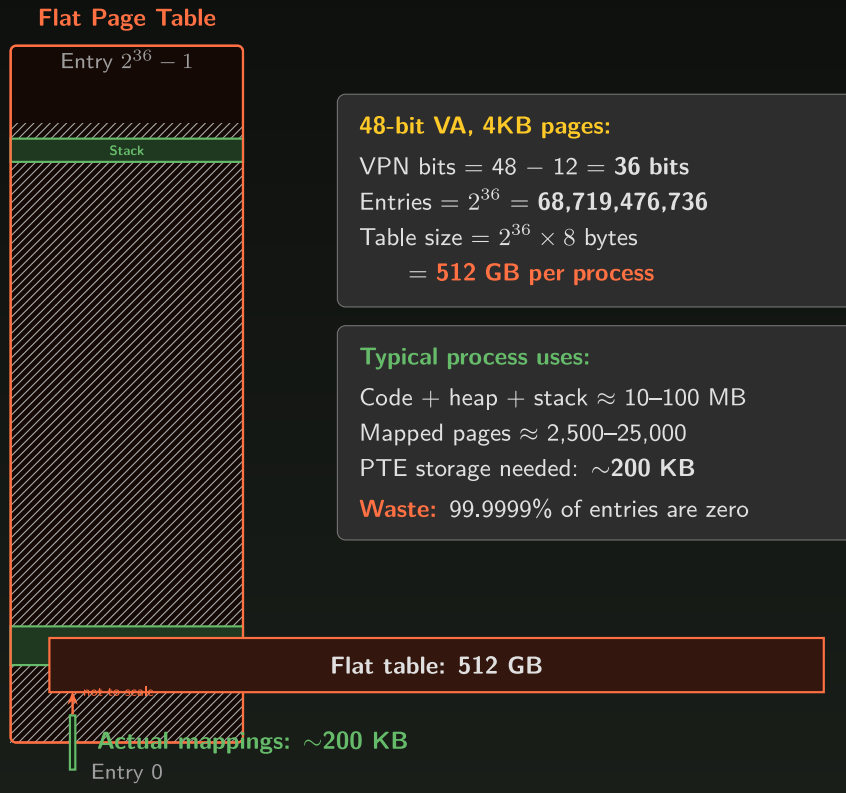
- AArch64 uses **48-bit virtual addresses** and by default **4KB pages**
- Page offset = 12 bits, so the Virtual Page Number (VPN) = 48 - 12 = **36 bits**
- Array size = 2entries x 8 bytes each = **512 GB per process**

Your process uses maybe 10 MB of memory. Its page table is 512 GB.

This is not a theoretical concern. It's physically impossible. You don't have 512 GB of RAM to store the page table, let alone the data it maps.

The Waste Visualized

Flat Page Table: One Entry Per Page



48-bit VA, 4KB pages:

VPN bits $= 48 - 12 =$ **36 bits**

Entries $= 2^{36} =$ **68,719,476,736**

Table size $= 2^{36} \times 8$ bytes

$=$ **512 GB per process**

Typical process uses:

Code + heap + stack $\approx$ 10–100 MB

Mapped pages $\approx$ 2,500–25,000

PTE storage needed: ~**200 KB**

Waste: 99.9999% of entries are zero

Attempt 2:

What if we limit virtual addresses to 34 bits (16 GB), enough for most processes?

- VPN = 34 - 12 = 22 bits
- Array = 2entries x 8 bytes = **32 MB per process**
- 100 processes x 32 MB = **3.2 GB** of RAM just for page tables

Better than 512 GB, but still terrible. A process with 10 MB of data spends 32 MB on its page table. And you've given up the sparse 48-bit address space.

A flat array allocates space for **every possible page**, not just the pages that are actually mapped.

Attempt 3: Split the VPN

Split the 22-bit VPN into two parts: a 13-bit **L1 index** and a 9-bit **L2 index**.

- **L1 directory**: 2= 8,192 entries x 8 bytes = **64 KB** (always allocated)
- **L2 table**: 2= 512 entries x 8 bytes = **4 KB** (allocated only when needed)

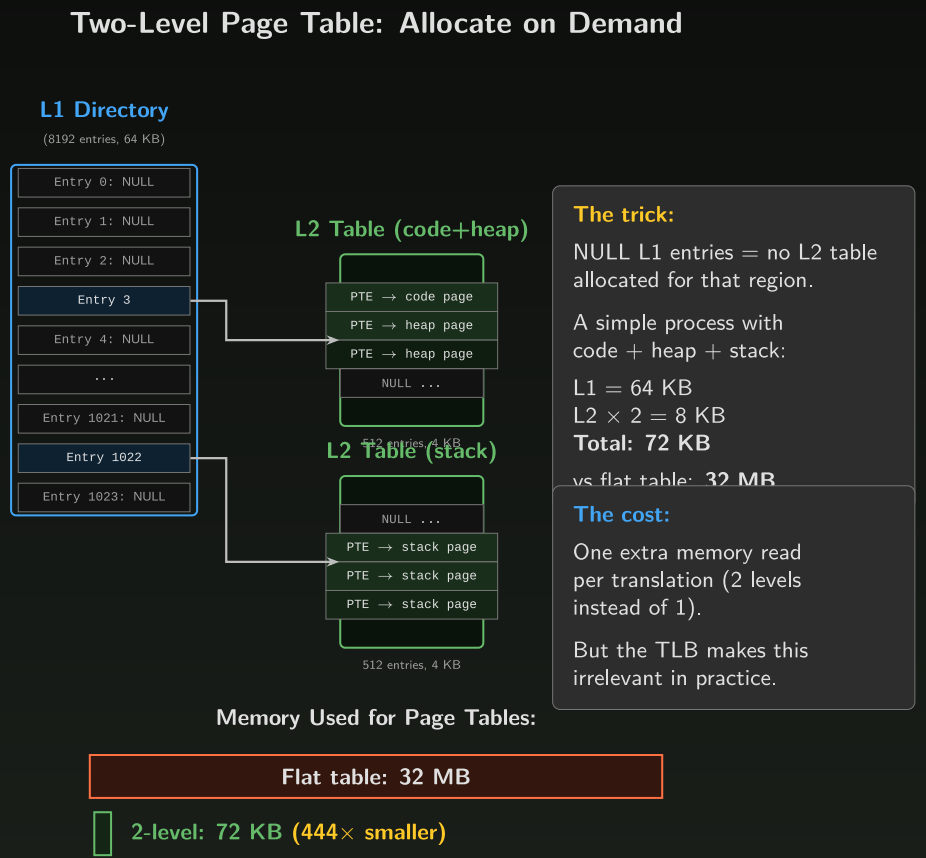
Each L1 entry either points to an L2 table or is NULL (unmapped region).

A simple process with code, heap, and stack touches maybe 2 L2 tables:

- L1 = 64 KB + L2 x 2 = 8 KB = **72 KB total**

That's way smaller than the flat 32 MB. And we didn't give up any address space. NULL L1 entries simply mean "no L2 table allocated for this region."

The Multi-Level Savings



Limitation

- Well,.... we only asusmed 16GB of physical RAM. What about modern systems that can have beyond 512GB of RAM

Scaling to 48-bit: Four Levels

Apply the same trick to a 48-bit VA with 4KB pages. VPN = 36 bits. Split into **four 9-bit indices**:

Level	VA bits	Index	Entries	Table size
L0	[47:39]	9 bits	512	4 KB
L1	[38:30]	9 bits	512	4 KB
L2	[29:21]	9 bits	512	4 KB
L3	[20:12]	9 bits	512	4 KB

Why 9?

- 512 entries x 8 bytes = 4096 bytes = **one 4KB page**.
- Each page table fits in exactly one physical page.
    - The granule size was chosen to make the table size equal the page size, so the allocator can hand out page tables one page at a time.
- $2^9 = 512$
- $2^{12} = 4096$

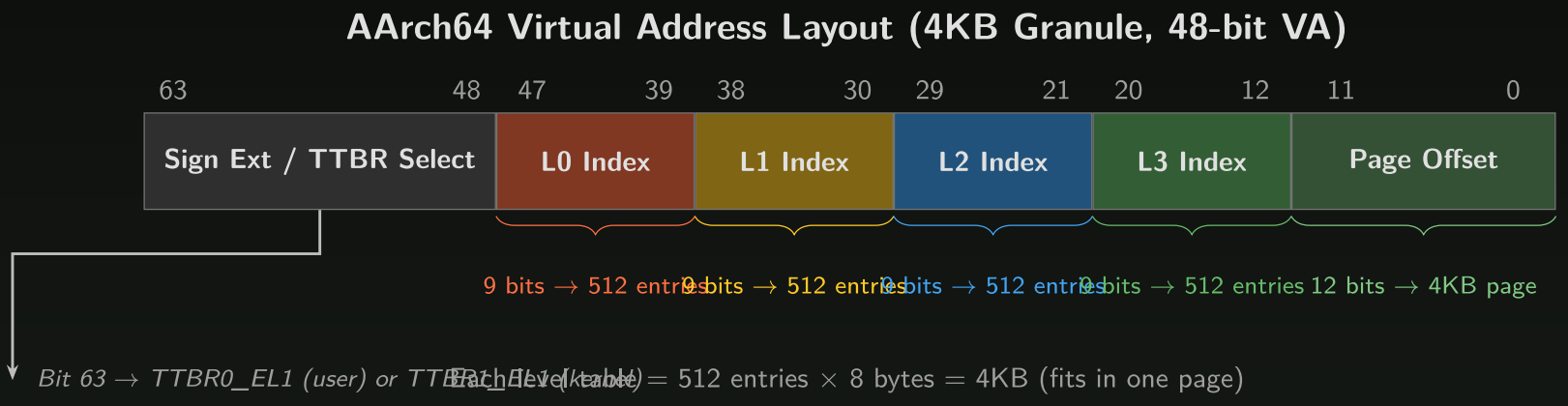
Granule Comparison

AArch64 Page Table Granule Comparison

	4 KB (Linux default)	**16 KB**	**64 KB**
Page offset bits	12 bits	14 bits	16 bits
Index bits / level	9 bits (512 entries)	11 bits (2048 entries)	13 bits (8192 entries)
Table size	4 KB (= 1 page)	16 KB (= 1 page)	64 KB (= 1 page)
Levels (48-bit VA)	**4** (L0–L3)	**4** (L0–L3)	**3** (L1–L3)
L0 entry maps	512 GB	128 TB	— (no L0)
Block descriptors	1 GB (L1), 2 MB (L2)	32 MB (L2)	512 MB (L2)
Min TLB entry maps	4 KB	16 KB	64 KB

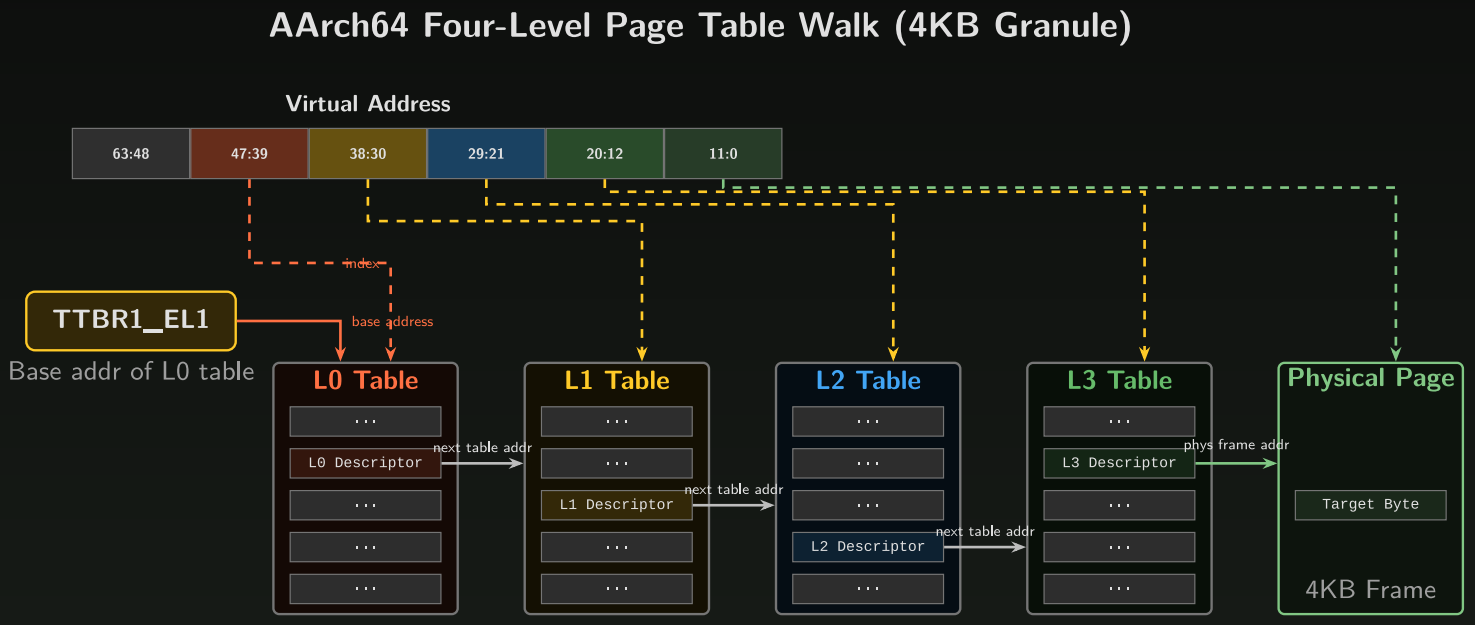
Table size always = one page = one allocation unit.
4KB granule: each table fits in a 4KB page → 512 entries × 8B = 4096B. **Not a coincidence.**

Virtual Address Bit Layout



- With 4KB granule and 48-bit VAs, the virtual address is split into 9-bit index fields plus a 12-bit page offset.
- Each 9-bit index selects one of 512 entries. Each entry is 8 bytes, so each table is exactly **512 x 8 = 4096 bytes = one 4KB page**.

The Hardware Page Table Walk



Solid arrows: Physical address from descriptor **Dashed arrows:** Index / offset from virtual address bits

Each table = 512 entries × 8 bytes = 4096 bytes = 1 page. Total addressable: $512^4 \times 4\,\text{KB} = 256\,\text{TB}$.

Block Descriptors: 4KB pages

L1 can map **1GB blocks**, L2 can map **2MB blocks**. The walk stops early. No further levels needed.

This matters for us:

- The kernel's linear map uses **2MB PMD section mappings** for all of physical RAM
- `sys_call_table` lives in this linear map region
- When `resolve_pte()` walks the page tables, it often hits a PMD section at L2, so there is no separate L3 PTE
- The `pmd_sect()` check catches this case and casts `pmd_t *` to `pte_t *`

Block descriptors use the **same permission bit layout** as page descriptors. AP bits are in the same position regardless of descriptor type. That's why the cast works.

What Each Level Maps

Level	VA bits	Index	Maps per entry	Linux name	Block possible?
L0	[47:39]	9 bits	512 GB	PGD (Page Global Directory)	No (not yet at least...)
L1	[38:30]	9 bits	1 GB	PUD (Page Upper Directory)	Yes (1GB huge page)
L2	[29:21]	9 bits	2 MB	PMD (Page Middle Directory)	Yes (2MB section)
L3	[20:12]	9 bits	4 KB	PTE (Page Table Entry)	No (leaf only)
Offset	[11:0]	12 bits	1 byte	-	-

Where do the Maps per Entry come from?

- $2 = 512$
- $4\text{KB} \times 512 = 2\text{MB}$
- $2\text{MB} \times 512 = 1\text{GB}$
- $1\text{GB} \times 512 = 512\text{GB}$
- Each level uses 9 bits and effectively subdivides from larger to smaller chunks

16KB Granule

Start from the granule size:

1. Page size = 16KB = 16384 = 2^{14} → **offset = 14 bits**
2. Each descriptor is 8 bytes → entries per table = 16384 / 8 = **2048 = 2^{11} → 11 index bits per level**
3. Remaining VA bits for 48-bit: 48 − 14 = 34. Three full levels use 33 bits, leaving **1 bit for L0** (only 2 entries!)

16KB Granule

Level	VA bits	Index	Maps per entry	Linux name	Block?
L0	[47]	1 bit	2 × 64GB = **128 TB**	PGD	No
L1	[46:36]	11 bits	2048 × 32MB = **64 GB**	PUD	No
L2	[35:25]	11 bits	2048 × 16KB = **32 MB**	PMD	Yes
L3	[24:14]	11 bits	**16 KB** (page)	PTE	No
Offset	[13:0]	14 bits	1 byte	—	—

Check: 1 + 11 + 11 + 11 + 14 = **48 bits** . L0 has only 2 entries because 34 bits don't divide evenly by 11. Only L2 supports block descriptors (32MB blocks).

64KB Granule

1. Page size = 64KB = 65536 = 2^{16} → **offset = 16 bits**
2. Each descriptor is 8 bytes → entries per table = 65536 / 8 = **8192 = 2^{13} → 13 index bits per level**
3. Remaining VA bits for 48-bit: 48 − 16 = 32. Two full levels use 26 bits, leaving **6 bits for L1** (only 64 entries). **No L0 at all — only 3 levels.**

64KB Granule

Level	VA bits	Index	Maps per entry	Linux name	Block?
L1	[47:42]	6 bits	64 × 512MB = **4 TB**	PGD (!)	No
L2	[41:29]	13 bits	8192 × 64KB = **512 MB**	PMD	Yes
L3	[28:16]	13 bits	**64 KB** (page)	PTE	No
Offset	[15:0]	16 bits	1 byte	—	—

Check: 6 + 13 + 13 + 16 = **48 bits** ✓ No L0 exists. L1 is the root but has a stunted 6-bit index (64 entries instead of 8192). Only L2 supports block descriptors (512MB blocks).

What's in a PTE?

Every 8-byte descriptor answers three questions:

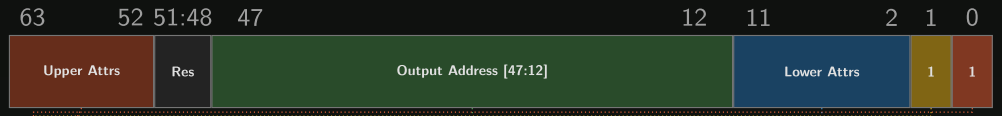
1. **Where?** Bits [47:12] hold the output address (next table base, or physical frame)
2. **Who can access?** AP[2:1] (bits [7:6]), UXN (bit 54), PXN (bit 53)
3. **Is it valid?** Bit [0]. If zero, the MMU generates a translation fault without reading any other field.

Bit [1] distinguishes **table** descriptors (points to next level) from **block/page** descriptors (leaf mapping). At L3, bit [1] = 1 always means a page descriptor.

The AP bits are what we care about for PTE manipulation.

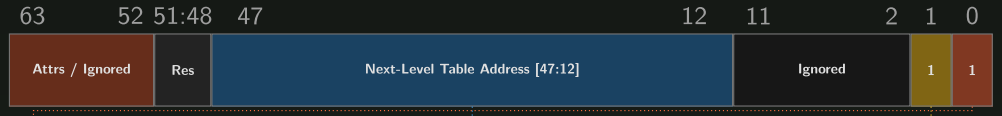
PTE Bitfield Layout

L3 Page Descriptor (4KB Granule)



- [63:52] Upper Attrs: PXN, UXN, Contiguous (software-defined)
- [47:12] Output Address: physical frame (36 bits, up to 256 TB)
- [11:2] Lower Attrs: AttrIndx, NS, AP[2:1], SH, AF, nG
- [1] Type: 1 = page (L3) or table (L0-L2), 0 = block
- [0] Valid: 1 = entry is valid, 0 = translation fault

L0/L1/L2 Table Descriptor (points to next-level table)



- [47:12] Next-level table physical address (4KB aligned)
- [1] Type: 1 = table descriptor, 0 = block descriptor
- [0] Valid: 1 = valid entry, 0 = fault

AP[2:1]: The Permission Table

AP[2:1]	EL1 (Kernel)	EL0 (User)
00	Read/Write	No Access
01	Read/Write	Read/Write
10	Read-Only	No Access
11	Read-Only	Read-Only

What to flip to modify Syscall table?

AP[2] = `PTE_RDONLY` (bit 7). This is the exact bit a rootkit flips.

- The kernel maps `sys_call_table` with AP = `10` (read-only at EL1, no access at EL0).
- clears bit 7 to change AP to `00` (read-write at EL1):

```
new = __pte(pte_val(saved_pte) & ~PTE_RDONLY);
```

One bit clear makes `.rodata` writable.

UXN, PXN, AF, and AttrIndx

Bit [54] UXN / Bit [53] PXN (User/Privileged Execute Never):

- Enforce W^X: writable pages are not executable, executable pages are not writable

Bit [10] AF (Access Flag):

- Set by hardware on first access. Used for page aging and LRU decisions

Bits [4:2] AttrIndx:

- Indexes into `MAIR_EL1` to select memory type (Normal cacheable, Device, Non-cacheable)

PAN: Privileged Access Never

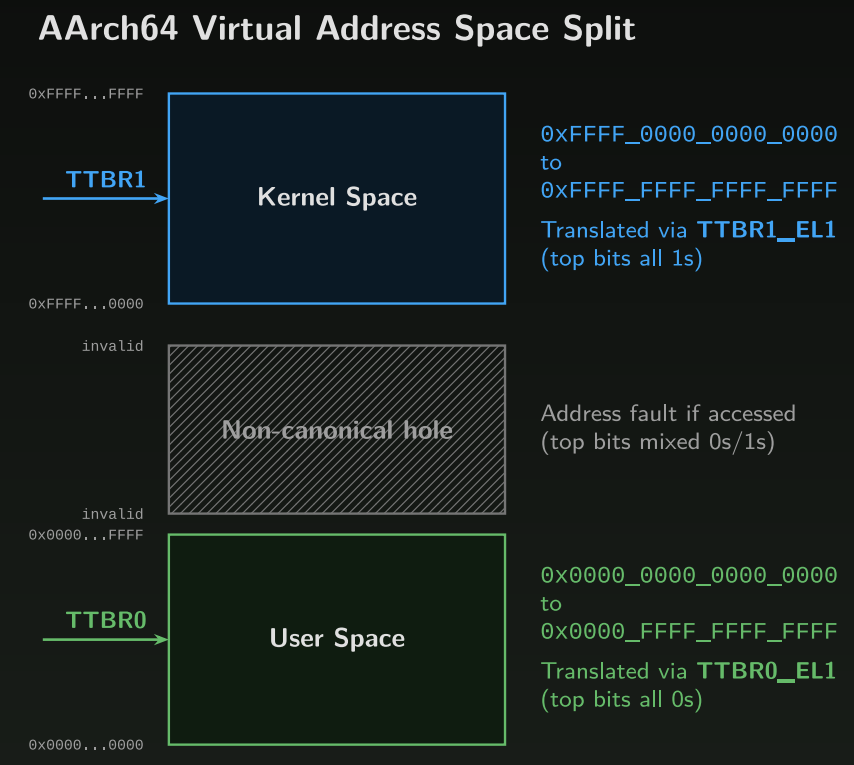
ARMv8.1 feature. When `PSTATE.PAN = 1`:

- EL1 (kernel) **cannot directly access userspace memory** (TTBR0 addresses)
- Any kernel load/store to a user address generates a permission fault
- Kernel must use `copy_from_user()` / `copy_to_user()` (which temporarily clear PAN)

This mitigates an entire class of exploits that rely on placing data at a user address and tricking the kernel into dereferencing it.

Linux enables PAN by default on all ARMv8.1+ cores.

Two TTBRs: The Address Space Split



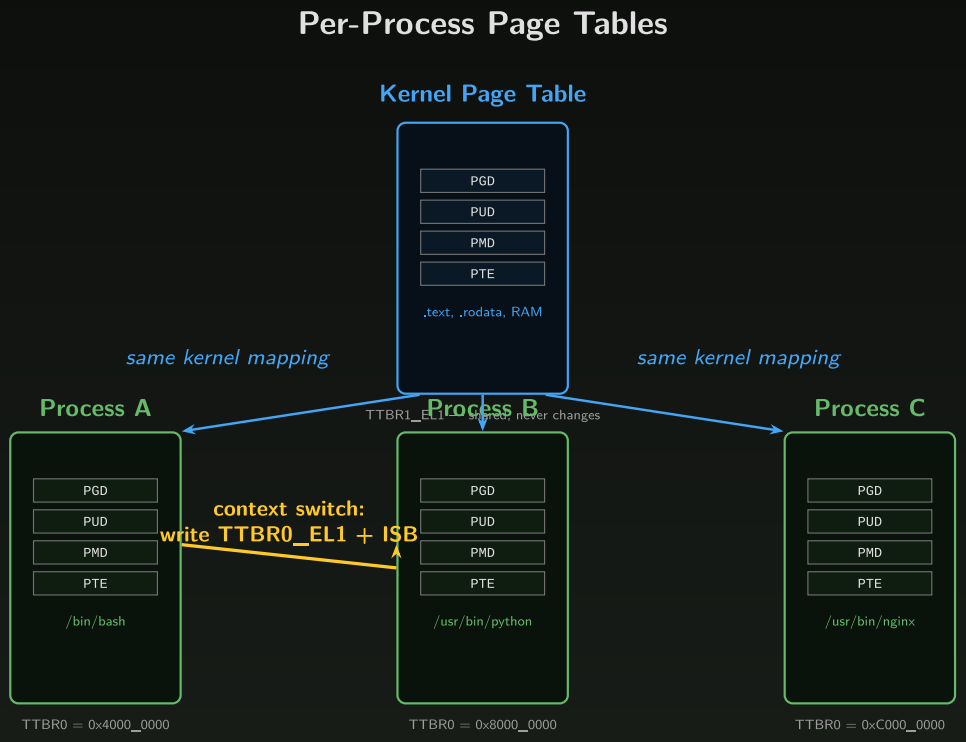
TCR_EL1.T0SZ and T1SZ control the size of each region
(commonly set for 48-bit VA → 256 TB per half)

two TTBSsL

AArch64 splits the 64-bit VA space into two halves using **two base registers**:

Register	Addresses	Top bits	Used for
TTBR0_EL1	`0x0000_0000_0000_0000` to `0x0000_FFFF_FFFF_FFFF`	All zeros	Userspace
TTBR1_EL1	`0xFFFF_0000_0000_0000` to `0xFFFF_FFFF_FFFF_FFFF`	All ones	Kernel

Per-Process Page Tables



Context switch = write new TTBR0_EL1 (user page table base + ASID).
TTBR1_EL1 **never changes**. The kernel mapping is identical in every process.

Context Switch

On a context switch, the kernel writes a new value to `TTBR0_EL1`:

```
MSR  TTBR0_EL1, X0     // new process's page table base + ASID
ISB                     // instruction barrier: pipeline sees new TTBR
```

`TTBR1_EL1` is **never touched**. The kernel mapping is identical in every process.

- The page table swap **is** the isolation mechanism.
- After the MSR, the old process's user pages are simply invisible. The MMU walks a different tree.

ASID: Avoiding Full TLB Flushes

Each TLB entry is tagged with an **Address Space Identifier** stored in `TTBR0_EL1[63:48]`.

On context switch, write new `TTBR0` with new ASID. Old entries remain but are invisible (wrong ASID tag). No flush needed.

`TCR_EL1.AS` selects 8-bit (256) or 16-bit (65,536) ASIDs. Linux prefers 16-bit.

What happens with 5,000 processes and 256 ASIDs?

- Generation rollover: kernel bumps a generation counter
- All TLB entries from the old generation become stale
- `TLBI VMALLE1` flushes everything, but only when the generation wraps

Feel the Pain: Why We Need the TLB

Without any translation cache, every memory access requires the MMU to read 4 page table levels from DRAM:

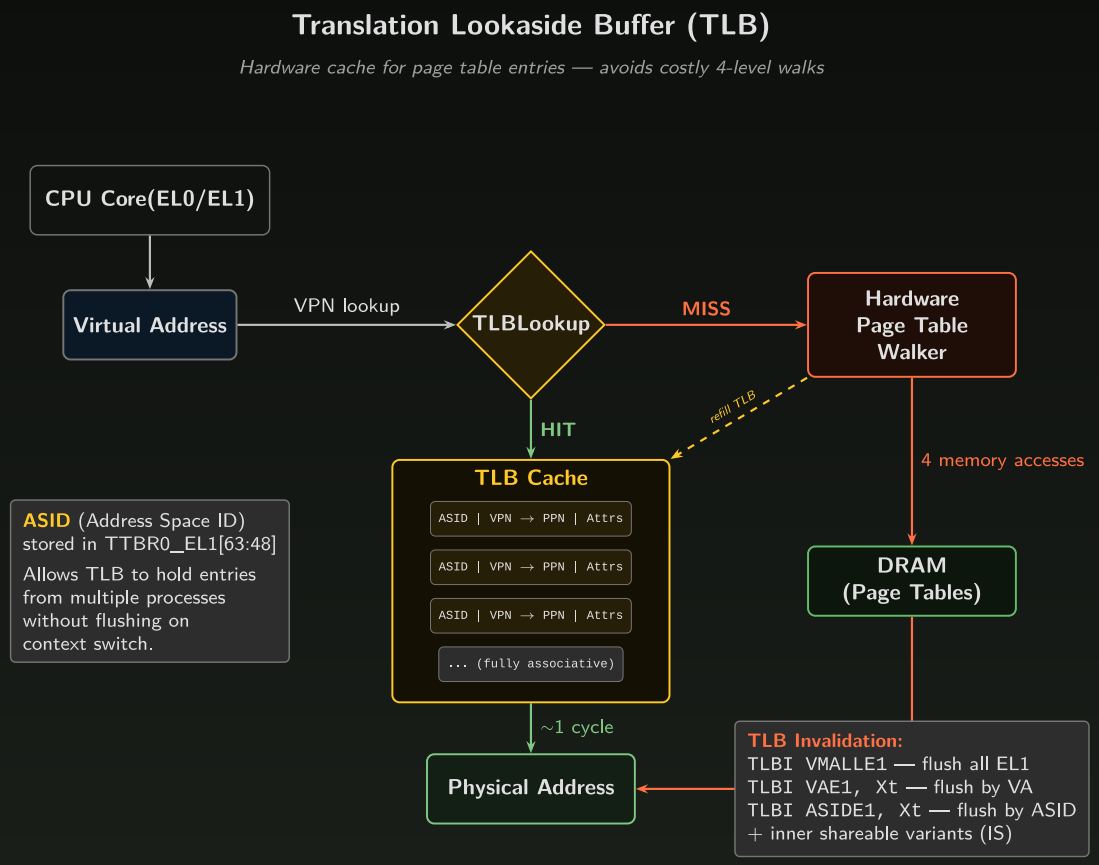
- 4 reads at ~100 ns each = about 400 ns per translation
- A normal access takes ~5 ns from L1 cache
- If we translate every address that will lead t significant overhead

Consider a simple loop:

```
for (int i = 0; i < N; i++)
    sum += arr[i];
```

`arr[i]` and `arr[i+1]` are on the **same 4KB page**. Why walk the page table again? The mapping hasn't changed. We need a cache for translations.

The TLB: Translation Lookaside Buffer



TLB Architecture

Modern AArch64 cores have multi-level TLBs:

Level	Size	Latency	Notes
L1 uTLB	~48 entries	1 cycle	Fully associative, per-type (I/D)
L2 Main TLB	~1024-5120 entries	5-10 cycles	Unified, set-associative
HW table walker	N/A	10-400+ cycles	Reads page tables from data cache / DRAM

Key features:

- **ASID tagging**: entries from different processes coexist, no full flush on context switch
- **VMID tagging**: under virtualization, entries from different VMs coexist
- **Hardware walker**: AArch64's MMU does the walk entirely in hardware, no software TLB refill trap

TLB Invalidation

When the kernel modifies a PTE, it **must** flush the stale TLB entry:

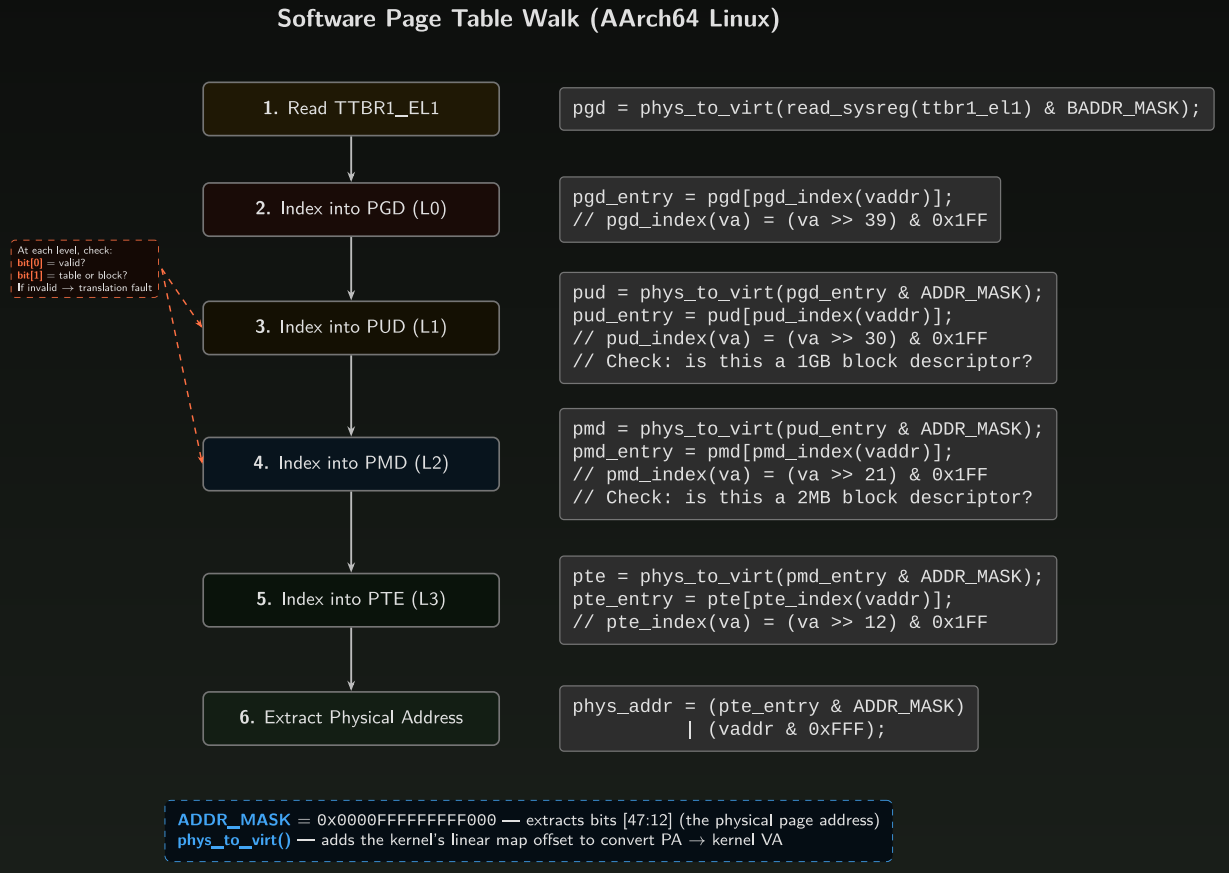
```
TLBI   VMALLE1IS        // Invalidate all EL1 entries on all cores
DSB    ISH              // Wait for invalidation to complete everywhere
ISB                     // Subsequent instructions see new mapping
```

The `IS` (Inner Shareable) suffix **broadcasts** invalidation to all cores. Without it, other cores continue using stale cached translations.

Why `flush_tlb_all()` is mandatory after PTE modification:

- Some CPUs still see the old **read-only** translation in their TLB
- Write to `sys_call_table` with stale TLB entry saying read-only = **permission fault = kernel crash**

Software Page Table Walk



The Walk in C

The walk uses Linux's page table API macros:

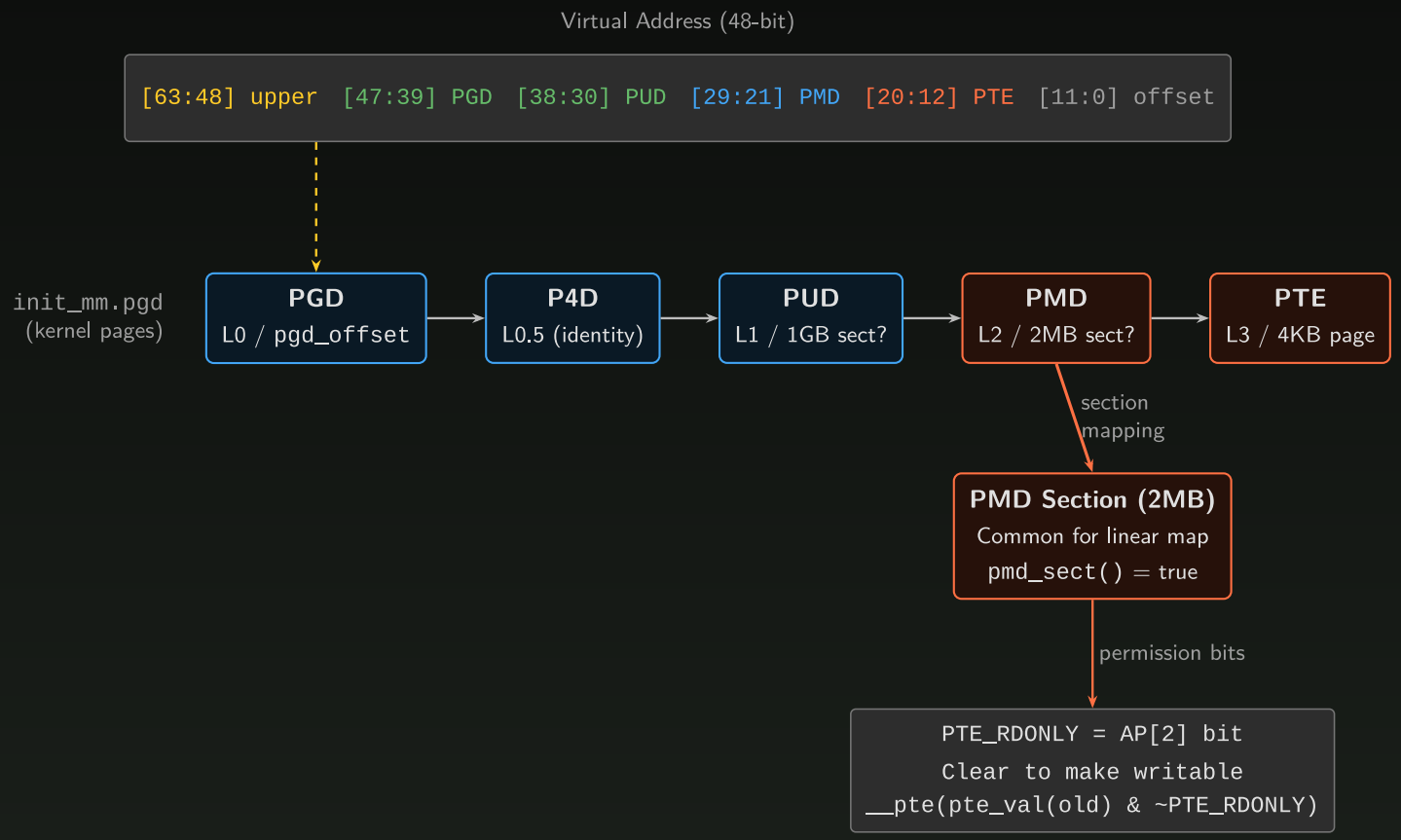
- `pgd_offset_k(va)`: start at `init_mm.pgd`
- `*_offset(parent, va)`: index into next level
- `*_none(entry)`: check valid bit (bit 0)
- `*_sect(entry)`: check block vs table (bit 1)

Each macro extracts the relevant 9 bits and indexes into the current table. If we hit a **PMD section** (2MB block), the walk stops early.

Walk the PT

- Walk from `init_mm.pgd` to `sys_call_table`'s PMD section.
- The kernel linear map uses 2MB block descriptors, so `resolve_pte()` returns a pointer to the PMD entry itself, cast to `pte_t *`.

Applied PTE Walk: sys_call_table



Clearing PTE_RDONLY

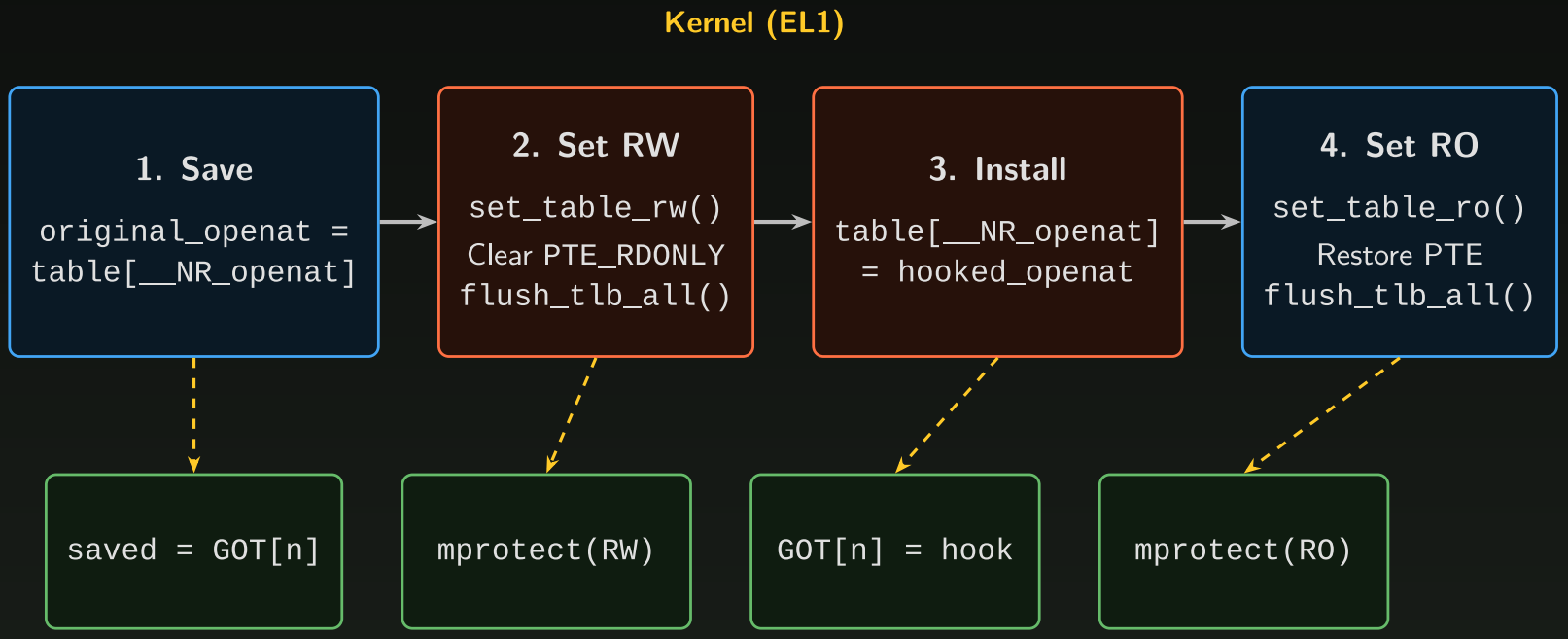
Sample helper: `set_table_rw()` Three steps: resolve, clear bit 7, flush TLB.

```
static int set_table_rw(void)
{
    unsigned long addr = (unsigned long)sys_call_table_ptr;
    unsigned int level;
    // you implement resolve_pte
    saved_ptep = resolve_pte(addr, &level);   /* 1. Walk page tables */
    if (!saved_ptep) return -EINVAL;
    saved_pte = READ_ONCE(*saved_ptep);
    /* 2. Clear PTE_RDONLY (AP[2], bit 7) */
    pte_t new = ...
    WRITE_ONCE(*saved_ptep, new);
    flush_tlb_all();                          /* 3. Broadcast TLB invalidation */
    return 0;
}
```

- After this returns, `sys_call_table_ptr[__NR_openat] = hook_fn` succeeds.
- Make sure to change the permissions back to restore protection.

Install hook

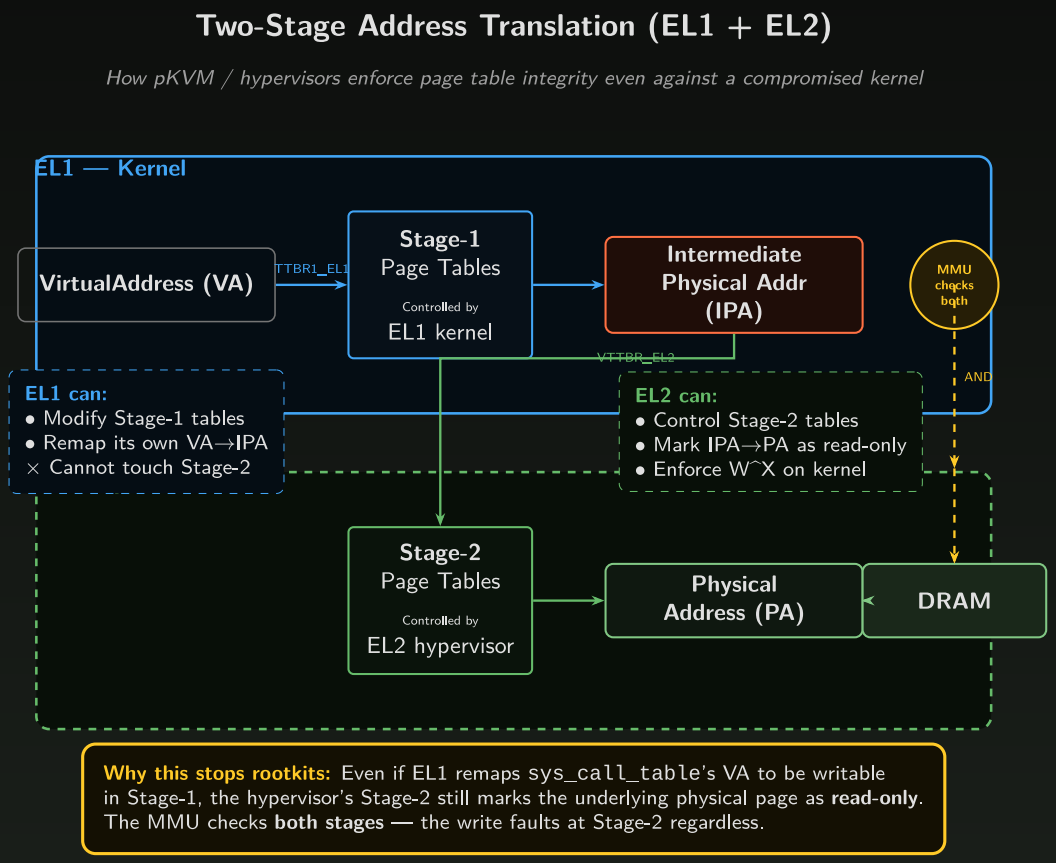
- implement helpers `set_table_rw/set_table_ro`



Mitigations

- "Maybe we shouldn't let the kernel modify page tables directly?"
- Easier said than done
- Whatever mechanism provides "security/integrity" for Exception Level i must be at exception level $j > i$
- Time for a quick side quest ...

Hypervisor --> PT Integrity



Hypervisor + PT Integrity

- With a hypervisor at EL2, the MMU performs two translation stages.
- Permissions are AND'd: both stages must grant access.
- **RW (Stage-1) AND RO (Stage-2) = RO.**

The Security Stack

Layer	What it does	Bypassable from EL1?
`.rodata` placement	Syscall table in read-only data segment	Yes (remap page tables)
PTE AP bits = RO	MMU enforces read-only on every access	Yes (rewrite the PTE)
KASLR	Randomizes kernel base address	Yes (read `kallsyms` from EL1)
Module signing	Prevents loading unsigned kernel modules	No (can't get code into EL1)
Kernel lockdown	Blocks `/dev/mem`, `kexec`, MSR access	No (can't get a write primitive)
Stage-2 (pKVM)	**Hypervisor marks physical pages RO**	**No, architecturally enforced**
PAN / PAC / BTI	Restrict kernel access, sign pointers, guard branches	Raises the bar

Page table Protections in Practice

- IOS: SPTM/PPL: No documented bypass
  - Trianguation doesnt count.
- Samsung Knox
- Android pKVM
- Windows: VBS

Discussion

- What else can you do with writable Page Tables?
- What if a driver gave you the ability to Read/write memory?
- How could this lead to code execution in a userland process?
- What about in the kernel itself?